

Базы данных

Тема 7

СУБД: Создание таблиц, категории целостности
данных

Целостность данных

- целостность данных (*data integrity*) – это гарантия точности, полноты и согласованности данных организации на любом этапе их жизненного цикла;
- цель обеспечения целостности данных:
 - гарантировать, что аналитика данных основана на достоверной информации;
 - гарантировать соответствие отраслевым стандартам и законодательным ограничениям;
- комплексный подход, затрагивающий технологическую инфраструктуру организации, политики и лиц, работающих с данными:
 - проверка на наличие ошибок;
 - процедуры валидации;
 - защита данных организации от потерь, утечек и искажений (шифрование, контроль доступа, резервное копирование);
- в базах данных обеспечивается, в том числе, ограничениями целостности данных (*integrity constraints*).

Категории целостности данных

- *сущностная целостность (entity integrity)*: определяет уникальность сущности в рамках хранимого набора данных;
- *доменная целостность (domain integrity)*: определяет достоверность значений в конкретном элементе данных (количество, формат и допустимые для ввода значения);
- *ссылочная целостность (referential integrity)*: обеспечивает наличие соответствующих взаимосвязей в связанных элементах данных, предотвращая появление потерянных (*orphaned*) данных при добавлении, изменении или удалении;
- *пользовательская целостность (user-defined integrity)*: определяет правила и ограничения для данных в соответствии с требованиями предметной области (бизнес-правила);
- *физическая целостность (physical integrity)*: обеспечивает целостность данных при их хранении и извлечении.

Сущностная целостность: реляционные базы данных

- определяет уникальность сущности в рамках хранимого набора данных, т.е. уникальность строки в конкретной таблице;
- предполагает обеспечение целостности столбцов идентификаторов или первичного ключа таблицы с помощью:
 - ограничений PRIMARY KEY;
 - ограничений UNIQUE;
 - индексов;
 - других средств, специфичных для конкретной СУБД.

Доменная целостность: реляционные базы данных

- определяет достоверность записей в конкретном столбце (формат и допустимые для ввода значения);
- доменная целостность обеспечивается:
 - определением типа данных: с любым объектом, содержащим данные (столбцы таблиц и представлений, переменные, параметры и результаты функций), связан тип, определяющий виды данных, которые могут храниться в этом объекте;
 - определением формата при помощи ограничений CHECK;
 - определением набора или диапазона возможных значений при помощи ограничений FOREIGN KEY, CHECK, DEFAULT, NOT NULL.

Ссылочная целостность: реляционные базы данных

- обеспечивает согласованность записей в связанных таблицах – предотвращаются следующие действия:
 - добавление или изменение записей в связанной таблице, если в первичной таблице нет соответствующей записи;
 - изменение значений в первичной таблице, которое приводит к появлению потерянных записей в связанной таблице;
 - удаление записей из первичной таблицы, если имеются совпадающие ключи в дочерних таблицах;
- ссылочная целостность, как правило, основана на связи первичных и внешних ключей и обеспечивается с помощью ограничений FOREIGN KEY.

Пользовательская целостность: реляционные базы данных

- определяет правила и ограничения для данных в соответствии с требованиями предметной области (бизнес-правила), дополняющие ограничения других категорий целостности;
- пользовательская целостность обеспечивается:
 - ограничениями уровня столбца и уровня таблицы (в инструкции CREATE TABLE);
 - средствами программирования, специфичными для конкретной СУБД (триггерами, функциями, хранимыми процедурами и т.п.).

СУБД SQL Server 2025 – Создание таблиц: типы данных, ограничения.

Выражения

- выражение - это сочетание идентификаторов, значений и операторов, которое SQL Server 2025 может вычислить для получения результата;
- выражения могут использоваться в различных контекстах во время доступа к данным или их изменения, например:
 - как часть данных, получаемых в запросе;
 - в качестве условий поиска данных, отвечающих определенному набору критериев;
- выражение может быть:
 - константой;
 - функцией;
 - именем столбца;
 - переменной;
 - вложенным запросом;
 - функцией CASE, NULLIF или COALESCE;
 - комбинацией перечисленных сущностей, соединенных операторами.

Категории операторов

Операции	Категория оператора
Сравнение значения с другим значением или выражением.	Операторы сравнения
Проверка истинности условия, такого как AND, OR, NOT, LIKE, ANY, ALL, IN.	Логические
Сложение, вычитание, умножение, деление и взятие остатка от деления.	Арифметические операторы
Выполнение действия над одним операндом, например положительным или отрицательным, или над дополнением.	Унарные
Временное обращение регулярных числовых значений в целочисленные и выполнение побитовой арифметической операции.	Битовые операторы
Постоянное или временное объединение двух строк (символьных или двоичных) в одну строку.	Оператор сцепления строк
Присваивание значения переменной или связывание столбца результирующего набора с псевдонимом.	Присвоение

Функции

- функции можно использовать и включать в следующие конструкции:
 - список выбора инструкции SELECT;
 - предложение WHERE инструкций SELECT, INSERT, DELETE или UPDATE для ограничения количества строк, удовлетворяющих условиям запроса;
 - условие поиска в предложении WHERE представления, которое должно динамически соответствовать пользователю или среде во время выполнения;
 - любое выражение;
 - ограничение CHECK или триггер для поиска указанных значений при вставке данных;
 - ограничение DEFAULT или триггер для вставки значений по умолчанию;

Категории функций

Категория функции	Описание
Статистические функции	Выполняют операции, объединяющие несколько значений в одно.
Функции настройки	Скалярные функции, возвращающие сведения о параметрах конфигурации.
Криптографические функции	Поддерживают шифрование, дешифрование, цифровые подписи и их проверку.
Функции для работы с курсорами	Возвращают сведения о состоянии курсора.
Функции даты и времени	Изменяют значения даты и времени.
Математические функции	Служат для выполнения тригонометрических, геометрических и других математических операций.
Функции метаданных	Возвращают сведения об атрибутах баз данных и объектов баз данных.
Ранжирующая функция	Недетерминированные функции, возвращающие ранжирующее значение для каждой строки секции.
Функции наборов строк	Возвращают результирующие наборы, которые можно использовать вместо ссылок на таблицу в инструкции Transact-SQL.
Функции безопасности	Возвращают сведения о пользователях и ролях.
Строковые функции	Изменяют значения char , varchar , nchar , nvarchar , binary и varbinary .
Системные функции	Работают или возвращают отчет о различных параметрах и объектах уровня системы.
Системные статистические функции	Возвращают сведения о производительности SQL Server.
Функции обработки текста и изображений	Изменяют значения text и image .

Категории функций: примеры

Категория функции	Описание
Статистические функции	AVG, MIN, CHECKSUM, SUM, CHECKSUM_AGG, STDEV, COUNT, STDEVP, COUNT_BIG, VAR, GROUPING, VARP, MAX
Функции конфигурации	@@OPTIONS, @@SERVERNAME, @@LANGUAGE, @@SERVICENAME, @@LOCK_TIMEOUT @@MAX_CONNECTIONS @@MAX_PRECISION @@VERSION @@NESTLEVEL,
Криптографические функции	SignByAsymKey, VerifySignedByAsmKey, SignByCert, ...
Функции для работы с курсорами	@@CURSOR_ROWS, CURSOR_STATUS, @@FETCH_STATUS
Функции даты и времени	DATEADD, DATEDIFF, DATENAME, DATEPART, DAY, GETDATE, GETUTCDATE, MONTH, YEAR
Математические функции	ABS, RAND, COS, EXP, ROUND, FLOOR, SIGN, ATAN, LOG, SIN
Функции метаданных	DB_NAME, OBJECT_NAME, DATABASEPROPERTY, FILEPROPERTY, ...
Ранжирующая функция	RANK, NTILE, DENSE_RANK, ROW_NUMBER
Функции наборов строк	CONTAINSTABLE, OPENQUERY, FREETEXTTABLE, OPENROWSET, OPENDATASOURCE, OPENXML
Функции безопасности	CURRENT_USER, SCHEMA_ID, USER_NAME, SUSER_ID, IS_MEMBER
Строковые функции	SOUNDEX, STR, REPLACE, LEFT, SUBSTRING, LEN, LOWER, LTRIM, ...
Системные функции	ISDATE, ISNULL, ROWCOUNT, NEWID, HOST_ID, CAST, CONVERT, ...
Системные статистические функции	@@CONNECTIONS, @@TOTAL_WRITE, @@CPU_BUSY, @@PACK_SENT, @@TIMETICKS, @@TOTAL_ERRORS, @@IO_BUSY, @@TOTAL_READ, ...
Функции обработки текста и изображений	PATINDEX TEXTVALID TEXTPTR

Функция CASE

- функция CASE - специальное выражение Transact-SQL, которое позволяет формировать альтернативные значения в зависимости от условия;
- функция CASE состоит из:
 - ключевого слова CASE;
 - имени преобразуемого столбца;
 - предложений WHEN, которые задают выражения для поиска, и предложений THEN, которые задают выражения для замены;
 - ключевого слова END;
 - необязательного предложения AS, задающего псевдоним функции CASE

```
SELECT ProductNumber, Name, 'Price Range' = CASE
WHEN ListPrice = 0 THEN 'Mfg item - not for resale'
WHEN ListPrice < 50 THEN 'Under $50'
WHEN ListPrice >= 50 and ListPrice < 250
                                THEN 'Under $250'
WHEN ListPrice >= 250 and ListPrice < 1000
                                THEN 'Under $1000'
ELSE 'Over $1000'
END [AS...]
FROM Production.Product
ORDER BY ProductNumber;
```

Типы данных

- с объектом, содержащим данные, связан тип, определяющий виды данных, которые могут храниться в объекте;
- типы данных имеют следующие объекты:
 - столбцы таблиц и представлений;
 - параметры хранимых процедур / функций;
 - переменные;
 - функции Transact-SQL, возвращающие одно или несколько значений;
 - хранимые процедуры, возвращающие значение (всегда имеет тип **integer**);
- можно создавать два вида пользовательских типов данных:
 - типы данных псевдонима - создаются на основе базовых типов данных (позволяют назначить типу данных имя, лучше характеризующее типы значений, которые будут храниться в объекте):

```
CREATE TYPE SSN FROM VARCHAR(11) NOT NULL;
```
 - пользовательские типы данных CLR основаны на типах данных, созданных в управляемом коде и помещенных в сборку SQL Server.

Атрибуты типов данных

- при назначении типа данных объекту определяются четыре атрибута объекта:
 - вид данных, содержащихся в объекте;
 - размер или длина хранимого объектом значения: количество байт, используемых для хранения числа;
 - точность: количество десятичных знаков;
 - масштаб числа: количество десятичных знаков справа от десятичного разделителя;
- точность (*precision*) и масштаб (*scale*) числовых типов данных, кроме **decimal** и **numeric**, фиксированы.

Встроенные типы данных

- ТОЧНЫЕ ЧИСЛОВЫЕ ТИПЫ:
 - tinyint, smallint, int, bigint;
 - bit;
 - decimal, numeric;
 - money, smallmoney;
- ВЕЩЕСТВЕННЫЕ ТИПЫ:
 - float, real;
- ДАТА И ВРЕМЯ:
 - date, time, datetime, datetime2, smalldatetime, datetimeoffset;
- СТРОКОВЫЕ ТИПЫ:
 - char(n), varchar(n), text;
 - Unicode: nchar(n), nvarchar(n), ntext;
- ДВОИЧНЫЕ СТРОКИ:
 - binary(n), varbinary(n), image;
- СПЕЦИАЛЬНЫЕ ТИПЫ:
 - geography, geometry - пространственные типы (spatial types);
 - hierarchyid;
 - json, xml;
 - vector;
 - rowversion;
 - sql_variant;
 - cursor;
 - table;
 - uniqueidentifier.

Числовые типы данных

Тип данных	Диапазон	Размер	
bit	0 или 1	кратно 1 байту	
bigint	-9 223 372 036 854 775 808 ... 9 223 372 036 854 775 807	8 байт	
int	-2 147 483 648 ... 2 147 483 647	4 байта	
smallint	-32 768 ... 32 767	2 байта	
tinyint	0 ... 255	1 байт	
decimal [(p[,s])] numeric [(p[,s])]	$-10^{38} + 1 \dots 10^{38} - 1$ точность $p=1\dots38$, по умолчанию $p=18$ масштаб $0 \leq s \leq p$, по умолчанию $s=0$	$p=1\dots9$	5 байт
		$p=10\dots19$	9 байт
		$p=20\dots28$	13 байт
		$p=29\dots38$	17 байт

битовые константы (bit)	0 или 1
целочисленные константы (int)	1894 -2 +15
константы decimal	1894.1204 2.0

Вещественные типы данных

Тип данных	Диапазон	Размер		
float [(<i>n</i>)]	$-1.79\text{E}+308 \dots -2.23\text{E}-308, 0,$ $2.23\text{E}-308 \dots 1.79\text{E}+308$ <i>n</i> =1...53 – количество битов, используемых для хранения мантиссы, фактически, <i>n</i> =24 или <i>n</i> =53 (по умолчанию)	<i>n</i> =1...24	7 разрядов	4 байта
		<i>n</i> =25...53	15 разрядов	8 байт
real	$-3.40\text{E}+38 \dots -1.18\text{E}-38, 0,$ $-1.18\text{E}-38 \dots 3.40\text{E}+38$	4 байта		

вещественные константы (float и real)	-12.34 101.5E5 0.5E-2
--	-----------------------------

Строковые (символьные) типы данных

Тип данных	Диапазон	Размер
char (<i>n</i>)	строковые данные фиксированного размера ($n = 1..8000$), значения дополняются пробелами, UTF-8 / кодовая страница по умолчанию	n байт $\leq n$ символов
varchar (<i>n max</i>)	строковые данные переменного размера ($n = 1..8000$; <i>max</i> – до 2 ГБ), UTF-8 / кодовая страница по умолчанию	$(n+2)$ байт $\leq n$ символов
nchar (<i>n</i>)	строковые данные фиксированного размера ($n = 1..4000$), значения дополняются пробелами	$2*n$ байт $\leq n$ символов
nvarchar (<i>n max</i>)	строковые данные переменного размера ($n = 1..8000$; <i>max</i> – до 2 ГБ)	$(n+2)$ байт $\leq n$ символов

символьные строки (кодовая страница по умолчанию)	'O'Brien' 'The level for job_id: %d should be between %d and %d.'
строки в Юникоде (UCS-2 / UTF-16)	N'Michél'

- по умолчанию $n = 1$ или $n = 30$ (функции CAST и CONVERT);
- правила сопоставления (collation) определяют кодировку (SBCS/MBCS, UCS-2, UTF-8, UTF-16) и правила сравнения символьных данных.

Двоичные типы данных

Тип данных	Диапазон	Размер
binary [(<i>n</i>)]	двоичные данные фиксированного размера ($n = 1..8000$), значения дополняются нулями	<i>n</i> байт
varbinary [(<i>n</i> max)]	двоичные данные переменного размера ($n = 1..8000$; max – до 2 ГБ)	(<i>n</i> +2) байт

константы двоичных строк	0x12Ef 0x69048AEFDD010E 0x
--------------------------	----------------------------------

- по умолчанию $n = 1$ или $n = 30$ (функции CAST и CONVERT).

Типы данных `uniqueidentifier`

Тип данных	Диапазон	Размер
<code>uniqueidentifier</code>	128-битовый глобально-уникальный идентификатор	16 байт

константы <code>uniqueidentifier</code>	'6F9619FF-8B86-D011-B42D-00C04FC964FF' 0xff19966f868b11d0b42d00c04fc964ff
---	--

- GUID (*globally unique identifier*), UUID (*universally unique identifier*) – 128-битный идентификатор, статистическая уникальность которого обеспечивается алгоритмом его вычисления (стандарты ITU-T Rec. X.667; ISO/IEC 9834-8; IETF RFC4122 / RFC 9562);
- вычисление нового значения в SQL Server 2025:
 - NEWID() – соответствует RFC4122;
 - NEWSEQUENTIALID().

Типы данных для даты и времени

Тип данных	Диапазон	Размер
date	0001-01-01 ... 9999-12-31	3 байта
time[(f)]	00:00:00.0000000 ... 23:59:59.9999999 <i>f=0..7 (по умолчанию f=7)</i>	3-5 байт
datetime2[(f)]	0001-01-01 ... 9999-12-31 00:00:00.0000000 ... 23:59:59.9999999 <i>f=0..7 (по умолчанию f=7)</i>	6-8 байт
smalldatetime	1900-01-01 ... 2079-06-06 00:00:00 ... 23:59:59 секунды округляются до минут	4 байта
datetimeoffset[(f)]	0001-01-01 ... 9999-12-31 00:00:00 ... 23:59:59.9999999 -14:00 ... +14:00 <i>f=0..7 (по умолчанию f=7)</i>	8-10 байт
datetime	1753-01-01 ... 9999-12-31 00:00:00 ... 23:59:59.997	8 байт

Константы для даты и времени

- формат ISO 8601 (не зависит от настроек языка / формата даты):
 - 'уууу-ММ-ддТНН:мм:сс[.nnnnnnnn][{+|-}hh:mm]'
 - 'уууу-ММ-ддТНН:мм:сс[.nnnnnnnn]Z'
 - '2004-05-23T14:25:10'
 - '2004-05-23T14:25:10.487'
 - '2007-05-08 12:35:29.1234567 +12:15'
- форматы, зависящие от настроек языка и/или формата даты/времени (SET LANGUAGE / SET DATEFORMAT):
 - алфавитный формат:
 - '20 Apr 2005'
 - '1996 APR[IL] [15]'
 - числовой формат (используются разделители '/', '-', ' или '.') :
 - '20/4/2005'
 - '04/15/98'
 - '04:24 PM'
 - '14:30:24'
 - строковый формат без разделителей:
 - '20050420'
 - '960420'

Преобразование типов данных

- тип выражения определяется типом входящих в него операндов;
- если, оператор соединяет два выражения разных типов, то производится приведение значений левой и правой частей к типу с высшим приоритетом согласно правилам предшествования типов (*data type precedence*) – по мере убывания приоритета: пользовательские типы, **json**, **sql_variant**, **xml**, **datetimeoffset**, **datetime2**, **datetime**, **smalldatetime**, **date**, **time**, **float**, **real**, **decimal**, **money**, **smallmoney**, **bigint**, **int**, **smallint**, **tinyint**, **bit**, **ntext**, **text**, **image**, **timestamp**, **uniqueidentifier**, **nvarchar** (включая **nvarchar(max)**), **nchar**, **varchar** (включая **varchar(max)**), **char**, **varbinary** (включая **varbinary(max)**), **binary**;
- если неявное приведение типов невозможно, выдается сообщение об ошибке;
- явное приведение типов:
 - `CAST ( expression AS data_type [ ( length ) ] )`
 - `CONVERT ( data_type [ ( length ) ] , expression [ , style ] )`
- очевидно, что в ряде случаев может быть невозможно как неявное, так и явное приведение типов.

Создание, изменение и удаление таблиц

- минимальный синтаксис инструкции CREATE TABLE требует указания имени новой таблицы, а также имен и типов данных каждого столбца (имена таблиц и столбцов должны соответствовать синтаксису идентификаторов)

```
CREATE TABLE products (  
    product_no integer,  
    name varchar(max),  
    price numeric  
) ON [PRIMARY];
```

```
ALTER TABLE doc_exc  
    ADD column_b VARCHAR(20) NOT NULL  
        CONSTRAINT exb_unique UNIQUE;
```

```
DROP TABLE Orders
```

Ограничение NOT NULL

- ограничение NOT NULL для столбца указывает, что столбец не может принимать неопределённое значение (NULL);
- данное ограничение явно задаётся в определении столбца, при этом также можно присвоить ему имя.

```
CREATE TABLE products (  
    product_no integer NOT NULL,  
    name nvarchar(50) NOT NULL,  
    price numeric(5,2)  
);
```

```
CREATE TABLE products (  
    product_no integer NOT NULL,  
    name nvarchar(50) CONSTRAINT products_name_not_null NOT NULL,  
    price numeric(5,2)  
);
```

Значения по умолчанию

- значение по умолчанию – значение, которое будет автоматически присвоено столбцу, если оно не будет явно указано при добавлении новой записи;
- если значение по умолчанию не объявлено явно, им считается значение NULL;
- значение по умолчанию может быть выражением, которое в этом случае вычисляется в момент присваивания значения по умолчанию.

```
CREATE TABLE products (  
    product_no integer,  
    name nvarchar(255),  
    price numeric DEFAULT 9.99,  
    added_at datetime2 DEFAULT getdate()  
);
```

Вычисляемые столбцы

- вычисляемый столбец – столбец, вычисляемый из других столбцов таблицы (не допускаются подзапросы);
- хранимый вычисляемый столбец вычисляется при записи (добавлении или изменении) и занимает место в таблице так же, как и обычный столбец.
- произвести запись непосредственно в генерируемый столбец нельзя.

```
CREATE TABLE dbo.tableX  
(  
    low INT,  
    high INT,  
    average AS (low + high) / 2,  
    delta AS (low + high) PERSISTED  
);
```

Ограничение CHECK

- ограничение CHECK – некоторое логическое выражение, вычисляемое при вставке или обновлении записи таблицы для проверки соответствия данных рассматриваемому ограничению;
- ограничение CHECK может быть определено:
 - на уровне отдельного столбца;
 - на уровне таблицы.

```
CREATE TABLE products (  
    product_no integer,  
    name nvarchar (255),  
    price numeric CHECK (price > 0),  
    discounted_price numeric,  
    CHECK (discounted_price > 0),  
    CONSTRAINT valid_discount CHECK (price > discounted_price)  
);
```

Ограничение первичного ключа

- ограничение первичного ключа подразумевает, что образующий его столбец или набор столбцов может быть уникальным идентификатором строк в таблице;
- значения входящих в первичный ключ столбцов должны быть уникальными в совокупности и каждое из них должно быть отлично от NULL.

```
CREATE TABLE dbo.Employee  
(  
    EmployeeID INT PRIMARY KEY CLUSTERED  
);
```

```
CREATE TABLE [dbo].[Supply](  
    [PartID] [int] NOT NULL,  
    [SupplierID] [int] NOT NULL,  
    [Price] [float] NOT NULL,  
    [DeliveryTime] [int] NOT NULL,  
    CONSTRAINT [PK_Supply] PRIMARY KEY CLUSTERED ([PartID],[SupplierID])  
);
```

Ограничение UNIQUE

- ограничение уникальности гарантирует, что данные в определённом столбце или группе столбцов уникальны среди всех строк таблицы;
- допускается наличие неопределённых значений (NULL) в столбцах, входящих в ограничение;
- допускается наличие нескольких ограничений UNIQUE в таблице.

```
CREATE TABLE Production.TransactionHistoryArchive4  
(  
    TransactionID int NOT NULL,  
    CONSTRAINT AK_TransactionID UNIQUE(TransactionID)  
);
```

```
ALTER TABLE Person.Password  
ADD CONSTRAINT AK_Password UNIQUE (PasswordHash, PasswordSalt);
```

```
ALTER TABLE dbo.DocExc  
DROP CONSTRAINT UNQ_ColumnB_DocExc;
```


Ограничение внешнего ключа

- ограничение внешнего ключа указывает, что значения столбца (или группы столбцов) должны соответствовать значениям в некоторой строке другой таблицы (ссылочная целостность);
- порядок, число и типы столбцов в ограничении должны соответствовать порядку, числу и типам целевых столбцов (целевые столбцы должны входить в ограничения PRIMARY KEY, UNIQUE или формировать уникальный индекс);
- таблица может содержать несколько ограничений внешнего ключа.

```
CREATE TABLE Sales.TempSalesReason (  
    TempID INT NOT NULL,  
    Name NVARCHAR(50),  
    CONSTRAINT PK_TempSales  
        PRIMARY KEY NONCLUSTERED (TempID),  
    CONSTRAINT FK_TempSales_SalesReason FOREIGN KEY (TempID)  
        REFERENCES Sales.SalesReason(SalesReasonID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

Действия для ограничений внешнего ключа

- предложения REFERENCES инструкций CREATE TABLE и ALTER TABLE поддерживают предложения ON DELETE и ON UPDATE:
 - [ON DELETE { NO ACTION | CASCADE | SET NULL | SET DEFAULT }]
 - [ON UPDATE { NO ACTION | CASCADE | SET NULL | SET DEFAULT }]
- по умолчанию подразумевается действие NO ACTION (указывает, что при попытке удалить или изменить строку с ключом, на которую ссылаются внешние ключи в строках других таблиц, нужно сообщить об ошибке, а для соответствующей инструкции DELETE / UPDATE выполнить откат);
- действия CASCADE, SET NULL и SET DEFAULT позволяют удалять и изменять значения ключей в таблицах, ссылающихся на таблицу, в которую вносятся изменения:
 - CASCADE: каскадное изменение ссылающихся таблиц;
 - SET NULL: установка NULL для ссылающихся внешних ключей;
 - SET DEFAULT: установка значений по умолчанию для ссылающихся внешних ключей.

Ограничения множественных каскадных действий

- последовательности каскадных ссылочных действий, запускаемые отдельными инструкциями DELETE или UPDATE, должны образовывать дерево без циклических ссылок;
- никакая таблица не должна появляться больше одного раза в списке всех каскадных ссылочных действий, вызванных инструкциями DELETE или UPDATE;
- кроме того, в дереве каскадных ссылочных действий к любой из задействованных таблиц должен быть только один путь;
- любая ветвь в дереве прерывается, как только встречается таблица, для которой указано действие NO ACTION или вообще не указано действие.

Создание столбцов идентификаторов: свойство IDENTITY

- при вставке значений в таблицу со столбцом идентификатора (столбцом, помеченным свойством IDENTITY), автоматически формируется следующее значение данного столбца (путём добавления значение шага приращения к текущему значению);
- использование свойства IDENTITY:
 - уникальность свойства IDENTITY гарантирована только в рамках таблицы, в которой оно использовано;
 - таблица может содержать только один столбец со свойством IDENTITY;
 - столбец должен иметь тип данных **decimal(p,0)**, **int**, **numeric**, **smallint**, **bigint** или **tinyint**;
 - можно указать начальное значение и шаг (по умолчанию, (1,1));
 - столбец идентификатора не должен допускать значений NULL и содержать определений или объектов по умолчанию.

Использование столбцов идентификаторов

- необходимо дополнительно определять ограничение уникальности с помощью PRIMARY KEY / UNIQUE;
- сгенерированные значения могут иметь пропуски (в результате отката транзакций или восстановления после сбоев сервера)
- сослаться на столбец в списке выборки можно создать с помощью указания ключевого слова \$IDENTITY, также сослаться на столбец можно при помощи имени;
- значение столбца идентификатора не может быть задано явно или как значение по умолчанию.

```
CREATE TABLE TZ (  
    Z_id INT IDENTITY(1,1) PRIMARY KEY,  
    Z_name VARCHAR(20) NOT NULL  
);
```

```
SELECT SCOPE_IDENTITY() AS [SCOPE_IDENTITY];
```

```
SELECT @@IDENTITY AS [@@IDENTITY];
```

```
SELECT IDENT_CURRENT('TZ') AS [IDENT_CURRENT];
```

Глобальные уникальные идентификаторы

- если в приложении нужно сформировать столбец идентификатора, уникальный для всей базы данных или всех баз данных во всех компьютерных сетях, необходимо использовать свойство ROWGUIDCOL, тип данных **uniqueidentifier** и функцию NEWID;
- использование свойства ROWGUIDCOL для определения столбца идентификаторов GUID:
 - в таблице может содержаться только один столбец ROWGUIDCOL, который должен иметь тип данных **uniqueidentifier**;
 - не обеспечивается уникальность значений, хранимых в столбце (необходимо использовать PRIMARY KEY / UNIQUE);
 - значение столбца не формируется автоматически – для вставки глобального уникального значения необходимо:
 - использовать для столбца определение DEFAULT с вызовом функций NEWID() / NEWSEQUENTIALID();
 - использовать вызов функций NEWID() / NEWSEQUENTIALID() в инструкции INSERT;
 - неизменность значения не контролируется;
 - ссылку на столбец в списке выборки, помимо использования имени, можно создать с помощью указания ключевого слова \$ROWGUID.

Использование столбцов глобальных идентификаторов

```
CREATE TABLE [dbo].[PurchaseOrderDetail]
(
    [LineNumber] [smallint] NOT NULL,
    [ProductID] [int] NULL REFERENCES Production.Product(ProductID),
    [UnitPrice] [money] NULL,
    [rowguid] [uniqueidentifier] ROWGUIDCOL NOT NULL
        CONSTRAINT [DF_PurchaseOrderDetail_rowguid] DEFAULT (newid()),
) ON [PRIMARY];
```

```
CREATE TABLE MyUniqueTable
(
    UniqueColumn UNIQUEIDENTIFIER DEFAULT NEWID(),
    Characters VARCHAR(10)
)
GO;
```

```
INSERT INTO MyUniqueTable(Characters) VALUES ('abc')
INSERT INTO MyUniqueTable VALUES (NEWID(), 'def')
```

Последовательности

```
CREATE SEQUENCE [schema_name . ] sequence_name
  [ AS [ built_in_integer_type | user-
        defined_integer_type ] ]
  [ START WITH <constant> ]
  [ INCREMENT BY <constant> ]
  [ { MINVALUE [ <constant> ] } | { NO MINVALUE } ]
  [ { MAXVALUE [ <constant> ] } | { NO MAXVALUE } ]
  [ CYCLE | { NO CYCLE } ]
  [ { CACHE [ <constant> ] } | { NO CACHE } ]
  [ ; ]
```

- в отличие от полей IDENTITY:
 - очередное значение последовательности можно получить с помощью функции NEXT VALUE FOR;
 - последовательность не привязана к конкретной таблице, следовательно, возможно обеспечить уникальность значений более чем в одной таблице;
 - для полей, сформированных с помощью последовательностей, не контролируются неизменность и уникальность значений.

Использование последовательностей

```
CREATE TABLE Test.Orders
```

```
(  
    OrderID int PRIMARY KEY,  
    Name varchar(20) NOT NULL,  
    Qty int NOT NULL  
);
```

```
GO
```

```
CREATE SEQUENCE Test.CountByOne
```

```
    START WITH 1
```

```
    INCREMENT BY 1;
```

```
GO
```

```
-- Insert records
```

```
INSERT Test.Orders (OrderID, Name, Qty)
```

```
VALUES (NEXT VALUE FOR Test.CountByOne, 'Tire', 2);
```

```
INSERT test.Orders (OrderID, Name, Qty)
```

```
VALUES (NEXT VALUE FOR Test.CountByOne, 'Seat', 1);
```

Вопросы к экзамену

- Основные элементы синтаксиса Transact-SQL: константы, встроенные функции, выражения.
- Типы данных: атрибуты, категории, применение. Приведение типов.
- Категории целостности данных.
- Создание, изменение и удаление таблиц: реализация сущностной целостности, вычисляемые столбцы.
- Создание, изменение и удаление таблиц: формирование значений суррогатных ключей.
- Создание, изменение и удаление таблиц: реализация доменной целостности, значения по умолчанию.
- Создание, изменение и удаление таблиц: реализация ссылочной целостности, действия для ограничений внешнего ключа.

Лабораторная работа №6.

Создание таблиц: ключи, ограничения, значения по умолчанию

1. Создать таблицу с автоинкрементным первичным ключом. Изучить функции, предназначенные для получения сгенерированного значения IDENTITY.
2. Добавить поля, для которых используются ограничения (CHECK), значения по умолчанию (DEFAULT), также использовать встроенные функции для вычисления значений.
3. Создать таблицу с первичным ключом на основе глобального уникального идентификатора.
4. Создать таблицу с первичным ключом на основе последовательности.
5. Создать две связанные таблицы, и протестировать на них различные варианты действий для ограничений ссылочной целостности (NO ACTION | CASCADE | SET NULL | SET DEFAULT).